

N-Queens Backtracking

1 Problem Definition

N-Queens Problem

Place N queens on an $N \times N$ chessboard so that no two queens attack each other.

No two queens may share the same row, column, or diagonal. The standard backtracking representation places exactly one queen in each row.

Representation

Let

$x[k]$ = column chosen for the queen in row k .

After placing queens in the first k rows, the partial solution is

$$(x[1], x[2], \dots, x[k]).$$

For example, the vector

$$[2, 4, 1, 3]$$

means:

$$Q_1 = (1, 2), \quad Q_2 = (2, 4), \quad Q_3 = (3, 1), \quad Q_4 = (4, 3).$$

2 Mathematical Safety Conditions

Assume queens have already been placed in rows

$$1, 2, \dots, k-1.$$

We want to test whether a new queen can be placed at row k , column c .

2.1 Column Condition

The new queen must not share a column with any earlier queen:

$$x[i] \neq c \quad \text{for every } 1 \leq i < k.$$

2.2 Diagonal Condition

Two squares $(i, x[i])$ and (k, c) lie on the same diagonal exactly when the row difference equals the column difference:

$$|i - k| = |x[i] - c|.$$

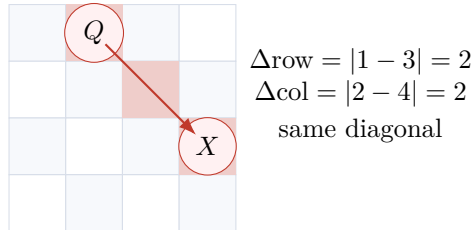
So the new queen is safe from diagonal attack only if

$$|x[i] - c| \neq |i - k| \quad \text{for every } 1 \leq i < k.$$

Complete Safety Test

A queen at row k , column c , is safe if

$$x[i] \neq c \quad \text{and} \quad |x[i] - c| \neq |i - k| \quad \text{for all } 1 \leq i < k.$$



3 Backtracking Search

Each search level corresponds to one row.

Level	Meaning
1	Choose the column for queen Q_1 .
2	Choose the column for queen Q_2 .
$\vdots$	Continue row by row.
N	Choose the column for queen Q_N . A full solution is formed.

Search Rule

Try columns 1 through N in the current row. Recurse only when the chosen column passes the safety test.

N-Queens

```
NQueens(k, n):
    for column = 1 to n:
        if Place(k, column):
            x[k] = column

            if k == n:
                print x[1..n]
            else:
                NQueens(k + 1, n)
```

Safety Test

```
Place(k, column):
    for i = 1 to k - 1:
        if x[i] == column:
            return false

        if abs(x[i] - column) == abs(i - k):
            return false

    return true
```

4 Worked Safety Check

Partial Board

Suppose the partial solution is

$$x[1] = 2, \quad x[2] = 4.$$

Test whether row 3, column 1, is safe.

The new candidate is

$$(k, c) = (3, 1).$$

Check against row 1:

$$x[1] = 2 \neq 1,$$

$$|x[1] - c| = |2 - 1| = 1, \quad |1 - 3| = 2.$$

Since $1 \neq 2$, there is no diagonal conflict with row 1.

Check against row 2:

$$x[2] = 4 \neq 1,$$

$$|x[2] - c| = |4 - 1| = 3, \quad |2 - 3| = 1.$$

Since $3 \neq 1$, there is no diagonal conflict with row 2.

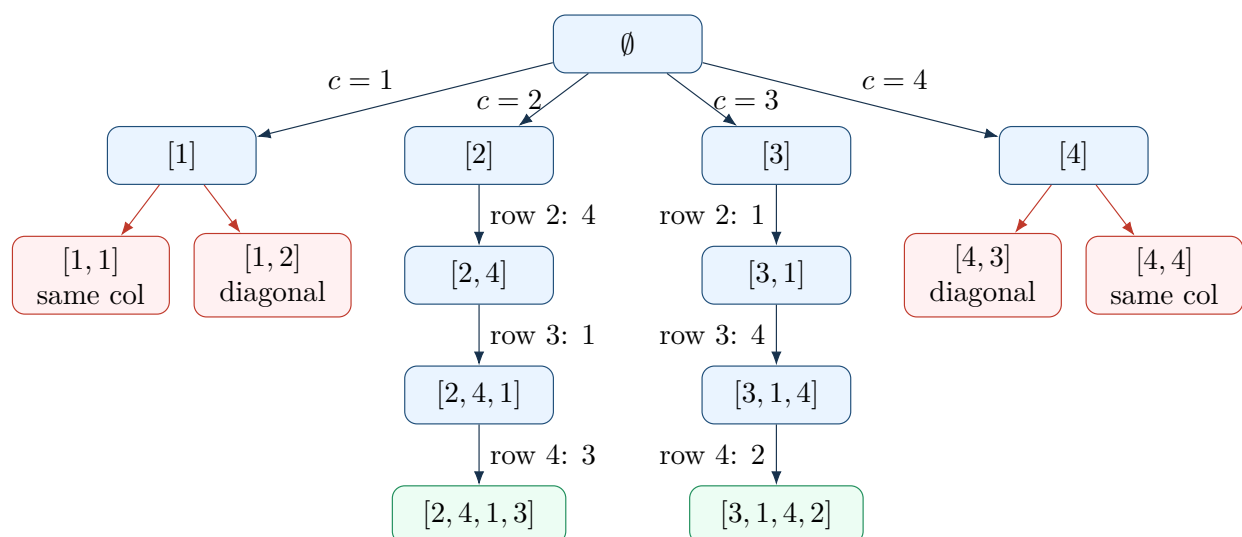
Result

Column 1 is safe for row 3. The partial solution becomes

$$[2, 4, 1].$$

5 State-Space Tree for $N = 4$

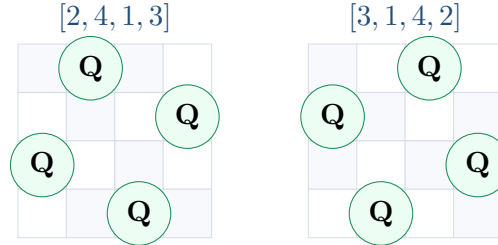
For the 4-Queens problem, many branches are killed by same-column and diagonal conflicts. The diagram shows a useful successful branch and some nearby pruned branches.



6 Two Solutions for $N = 4$

The two standard 4-Queens solutions are

$$\boxed{[2, 4, 1, 3]} \quad \text{and} \quad \boxed{[3, 1, 4, 2]}.$$



Mirror Property

The two 4×4 solutions are mirror images. Reflecting $[2, 4, 1, 3]$ horizontally gives $[3, 1, 4, 2]$.

7 How Backtracking Moves

If a queen is safely placed in row k , the algorithm moves to row $k + 1$. If row $k + 1$ has no safe column, the algorithm returns to row k , removes the old queen, and tries the next column.

Recursive Meaning

$$\text{NQueens}(k, n) = \text{search all safe completions from row } k \text{ to row } n.$$

8 Constructive Pattern: Even-Then-Odd Columns

Backtracking is the general method used in these notes. The source also mentions a direct construction that avoids search for many board sizes.

Main Idea

Place queens row by row using all even-numbered columns first, then all odd-numbered columns.

Even-Then-Odd Order

For a board of size N , define

$$E_N = (2, 4, 6, \dots, 2\lfloor N/2 \rfloor, 1, 3, 5, \dots, 2\lceil N/2 \rceil - 1).$$

Then place the queen in row k at column

$$x[k] = E_N[k].$$

Examples

$$E_4 = (2, 4, 1, 3), \quad E_5 = (2, 4, 1, 3, 5), \quad E_6 = (2, 4, 6, 1, 3, 5).$$

For $N = 6$, this means

$$x[1] = 2, \quad x[2] = 4, \quad x[3] = 6, \quad x[4] = 1, \quad x[5] = 3, \quad x[6] = 5.$$

When the Simple Pattern Works

The simple even-then-odd order gives a valid N-Queens placement when

$$N \bmod 6 \notin \{2, 3\}.$$

Equivalently, it works when the remainder after dividing N by 6 is 0, 1, 4, or 5.

Exceptional Cases

When

$$N \bmod 6 = 2 \quad \text{or} \quad N \bmod 6 = 3,$$

the simple even-then-odd order can create diagonal conflicts. In those cases, the construction must be modified, or the solution can be found using backtracking.

Why Same-Block Conflicts Do Not Occur

First consider two queens inside the even-column block. If their rows are $i < j$, then their row difference is

$$j - i.$$

Their columns are $2i$ and $2j$, so their column difference is

$$|2j - 2i| = 2(j - i).$$

Since

$$2(j - i) \neq j - i,$$

they cannot be on the same diagonal.

The same reasoning applies inside the odd-column block, because odd columns also increase by 2. Therefore:

No diagonal conflict occurs within the even block or within the odd block.

Where Conflicts Can Still Occur

The only possible diagonal conflict is between one queen from the even block and one queen from the odd block.

Let

$$h = \lfloor N/2 \rfloor.$$

An even-column queen in row i has column $2i$. An odd-column queen in row $h + t$ has column $2t - 1$.

These two queens lie on the same diagonal exactly when

$$h + t - i = |2i - (2t - 1)|.$$

Writing

$$a = i - t,$$

the dangerous case becomes

$$h - a = 2a + 1.$$

So

$$3a = h - 1.$$

Thus a conflict occurs exactly when $h - 1$ is divisible by 3. This gives the exceptional congruence classes

$$N \bmod 6 = 2 \quad \text{or} \quad N \bmod 6 = 3.$$

Takeaway

The even-then-odd construction is a useful shortcut, but it is not a complete replacement for backtracking. Use it directly only when

$$N \bmod 6 \notin \{2, 3\}.$$

9 Correctness Intuition

The algorithm is correct because it tries every possible column in each row, but only continues through safe partial boards.

1. If a full vector $[x[1], \dots, x[N]]$ is printed, every pair of queens passed the column and diagonal tests, so the board is valid.
2. If a valid board exists, none of its prefixes violates the safety test. Therefore, backtracking never prunes that valid path.

Conclusion

Backtracking prints exactly the valid N-Queens placements: no invalid placement is printed, and no valid placement is lost by pruning.

10 Complexity

The naive upper bound is N^N , because each of N rows may try N columns. With the column restriction, the search is closer to the permutations of columns:

$$N \cdot (N - 1) \cdot (N - 2) \cdots 1 = N!.$$

Each safety test may compare with up to $N - 1$ previous rows, so a simple implementation has worst-case cost bounded by

$$\mathcal{O}(N \cdot N!)$$

for enumerating the search tree with safety checks.

The recursion depth is N , and the solution vector stores N column values:

$$\text{space} = \mathcal{O}(N)$$

excluding the space used to print or store all solutions.

11 Final Review

N-Queens Essentials

$x[k]$ = column of queen in row k .
 $x[i] \neq x[k]$ prevents column conflicts.
 $|x[i] - x[k]| \neq |i - k|$ prevents diagonal conflicts.
Backtracking searches row by row and prunes unsafe prefixes.